ПП ПМ.12 · Раздел 1

Программирование мобильных игр

1

Результат

Разработать одну завершённую 2D-игру назначенного варианта: три проходимых уровня с ростом сложности, главное меню, выбор доступного уровня, игровые экраны, пауза, победа, поражение, настройки звука, рекорды и сохранение прогресса. Сохранить номер, название и особое правило из отдельного списка предметных областей.

2

Исходные условия

Использовать собственный работающий прототип УП ПМ.12 этого же модуля: управление, объекты, особое правило, победа, поражение и повтор. Скопировать проект перед изменениями. Связь относится только к ПМ.12. Если прототип утрачен, получить его из собственного репозитория или резервной копии; не заменять чужим готовым вариантом. Проверить его исходные функции по маршруту ниже до расширения.

3

Среда и порядок

Unity 6.6 (6000.6.2f1), C#, редактор кода, Git и Word. Последовательность: сохранить исходный проект → описать данные трёх уровней → реализовать состояния и переходы → подключить сохранения → проверить общие и индивидуальные случаи → исправить ошибки и сравнить Profiler → подготовить отчёт и репозиторий. Отдельные стенды вариантов не нужны: геометрию и данные создаёт студент в своём проекте.

ПП ПМ.12 · Раздел 2

Обязательные функции

1

Уровни и особое правило

Уровни должны различаться указанными в карточке числами, последовательностями, вместимостью, адресатами либо условиями проходов. Усложнять маршрут и ресурс, сохраняя достижимость цели. Перенести правило в данные уровня; не заменять его подсчётом любых контактов. Сохранять собственное оформление; отдельный проект оформления не требуется.

2

Экраны и управление

Реализовать Menu, Playing, Paused, Won и Lost. Кнопки: начать доступный уровень, пауза, продолжить, повторить, в меню, следующий уровень, настройки и назад. Закрытый уровень не запускается; поражение ничего не открывает; после третьего уровня нет кнопки перехода к четвёртому. Клавиатурное и экранное управление прекращаются при отпускании, паузе и смене состояния.

3

Состояние и данные

При новой попытке восстановить объекты и исходные параметры, обнулить груз, порядок, флаги, счётчики и таймер. Между запусками хранить только наибольший открытый уровень, лучший результат каждого уровня и громкость. Худший результат не перезаписывает лучший. Потеря фокуса включает паузу; возврат требует кнопки «Продолжить». Незавершённая попытка не восстанавливается.

ПП ПМ.12 · Раздел 3

Приёмочные проверки

1

Запуск и доступ

T01: при первом запуске доступен только уровень 1. T02: закрытый уровень не запускается. T03: выбранный уровень содержит свои объекты и условия. T04: выход до выполнения цели не даёт победу.

2

Исходы и повтор

T05: победа открывает ровно следующий уровень. T06: поражение не открывает уровень. T07: повтор очищает предметы, счётчики, таймер и состояние особого правила. T08: после уровня 3 показывается окончание игры.

3

Пауза и данные

T09: на паузе не меняются таймер, положение и счёт. T10: потеря фокуса включает паузу. T11: возврат фокуса требует «Продолжить». T12: после остановки и нового Play сохранены доступ, рекорд и громкость.

4

Индивидуальные проверки

T13: выполнить проверку из карточки варианта. T14: повторить её на трёх уровнях. T15: проверить минимум два размера Game View. T16: после повторных переходов не накапливаются объекты и ошибки. В таблице указать ожидаемый и фактический результат.

ПП ПМ.12 · Раздел 4

Проверка и сдача

1

Функциональная проверка

Выполнить T01–T16 и все особые проверки карточки на трёх уровнях. Записать исходное состояние, действия, ожидаемый и фактический результат, статус и доказательство. Проверить Console, повторные переходы и два размера Game View. Для Profiler сравнить одинаковый игровой эпизод до и после одного обоснованного изменения; не выдавать предполагаемые измерения за фактические.

2

Документы и доказательства

Заполнить «Отчет о выполнении.docx»: исходные условия, этапы и итоговая проверка. Вставить собственные рисунки меню, трёх уровней, паузы, победы, поражения, настроек и сохранённого прогресса после перезапуска; для индивидуального правила и Profiler сохранить сравнения до и после. В самом DOCX указать рабочую ссылку на репозиторий и хеш проверенного коммита. Установленные официальные формы практики заполнять только по действующему назначению, без подписей за других людей.

3

Передача результата

Передать DOCX, доступную преподавателю ссылку на репозиторий и хеш итогового коммита через установленный канал сдачи. В репозитории: Assets со всеми .meta, Packages, ProjectSettings, README и .gitignore; без Library, Temp, Logs, obj, UserSettings и Build. Клонировать новую копию и повторить запуск, победу, повтор и сохранение. Демонстрация 3–5 минут: правило, переход уровня, пауза, меню и перезапуск. Готовую сборку приложить только после фактической проверки; Android необязателен.

ПП ПМ.12 · Раздел 5

Файлы для работы

↓

Скачивания

Скачать контрольный пример «Кристальный маршрут» (ZIP)

Скачать форму отчёта (DOCX)

1

Основной проект

Работать в копии собственного прототипа ПМ.12. Для утраченной копии: открыть собственный репозиторий, нажать Code → Download ZIP, распаковать, найти каталог с Assets, Packages и ProjectSettings; добавить именно его в Unity Hub через Add → Add project from disk. Для сохранения истории предпочтительнее git clone по разделу сдачи.

2

Образец для чтения и сверки

«Кристальный маршрут» показывает устройства уровней, экранов и сохранений. Его полный код приведён далее и доступен отдельными файлами. Контрольный проект — готовая демонстрация другой области для необязательной сверки; он не является исходником индивидуальной работы или отдельной обязательной работой.

3

Форма отчёта

Скачать «Отчет о выполнении.docx», сохранить рабочую копию с тем же именем. Заполнять только собственные данные, действия и фактические проверки.

Как использовать разбор

1

Одна индивидуальная работа

Образец «Кристальный маршрут» объясняет общий приём. Читать его код и сопоставлять методы с собственным проектом. Создавать вторую игру образца не требуется. Разделы подготовки контрольной сцены можно использовать только для необязательной сверки; дальнейшие этапы выполнять в собственной копии.

2

Граница входа

До расширения должны работать движение, особое правило, ранний выход, победа, поражение и повтор. После расширения добавляются уровни, общие состояния, рабочие кнопки и сохранения. Если входная проверка не проходит, сначала восстановить соответствующий обработчик в собственном прототипе по разделам адаптации.

3

Что создаёт код образца

WorldBuilder строит камеру и LevelObjects; MenuController создаёт Canvas, EventSystem и панели. В индивидуальном проекте допустимы собственные сцены и UI; не запускать вторую систему экранов поверх существующей. Все 15 листингов образца доступны целиком в приложении.

ПП ПМ.12 · Раздел 7

01. Подготовьте рабочую копию

1

Что это и зачем

Рабочая копия содержит сцену, ресурсы и настройки собственного прототипа ПМ.12. Входная проверка отделяет имеющиеся ошибки от изменений текущей практики.

2

Действия

1. Остановить Play: верхняя кнопка Play не должна быть подсвечена.
2. Закрыть проект Unity.
3. Скопировать папку своего проекта в новую папку PP12_ВариантNN.
4. В Unity Hub нажать Add → Add project from disk.
5. Выбрать новую папку.
6. Открыть её редактором 6000.6.2f1.
7. В Project открыть Assets → Scenes → Main.
8. Выбрать File → Save.

3

Проверка

Открыта отдельная копия; исходная работа сохранена.

4

Если результат отличается

Если Hub сообщает о другой версии, выбрать установленную 6000.6.2f1. Не обновлять исходный проект без копии.

5

До изменения

В Game View проверить движение и отпускание, одно правильное и одно неправильное действие, выход до цели, победу, поражение и повтор. Записать фактическое состояние в разделе 1 отчёта. Если правило нарушено, не наращивать уровни: восстановить исходную версию из собственного Git или исправить обработчик, повторить тот же случай.

ПП ПМ.12 · Раздел 8

Подготовка контрольной сцены при необходимости

1

Что это и зачем

Этот раздел нужен только для необязательной сборки контрольного примера. Основная работа продолжается в копии собственного прототипа; второй проект не входит в сдачу. Сцена — сохранённый набор объектов; Hierarchy показывает объекты, Project — файлы.

2

Действия

1. В Unity Hub создать новый проект редактором 6000.6.2f1 с шаблоном 2D Built-In Render Pipeline.
2. Назвать проект PP12_CrystalRoute_Study.
3. Открыть проект.
4. В Project создать папку Assets/Scenes.
5. Создать папку Assets/Scripts.
6. Сохранить текущую сцену как Assets/Scenes/Main.unity.
7. В Hierarchy выделите Main Camera.
8. Удалить Main Camera из этой учебной сцены.
9. Сохранить сцену.

3

Проверка

Hierarchy пуст; в Project есть Scenes/Main и Scripts. Готовой игры пока нет. Камеру позднее создаст WorldBuilder.BuildCamera.

4

Если результат отличается

Не удалять объекты своей принятой работы: эти действия относятся только к новому учебному проекту.

ПП ПМ.12 · Раздел 9

03. Подготовьте ввод и UI

1

Что это и зачем

Unity UI (uGUI) предоставляет Canvas и кнопки. EventSystem передаёт им нажатия. Пример использует прежнюю систему ввода Unity: Input Manager.

2

Действия

1. Открыть Window → Package Management → Package Manager.
2. Найти пакет Unity UI (uGUI).
3. Если пакет отсутствует, установить его из Unity Registry.
4. Открыть Edit → Project Settings → Player.
5. Найти Active Input Handling в Other Settings.
6. Установить Input Manager (Old) или Both.
7. Если Unity запросит перезапуск редактора, выполнить его.

3

Проверка

Пакет uGUI установлен, выбран Input Manager (Old) или Both. После добавления скриптов MenuController создаст Canvas и EventSystem при запуске.

4

Если результат отличается

При ошибке UnityEngine.UI проверить пакет uGUI. При ошибке обращения к UnityEngine.Input проверить Active Input Handling.

ПП ПМ.12 · Раздел 10

04. Добавить полный набор C#-файлов

1

Что это и зачем

Скрипт — текст программы на C#. Компонент — скрипт, прикреплённый к объекту. Файлы примера ссылаются друг на друга, поэтому запуск возможен после добавления всего набора.

2

Действия

1. Открыть Assets/Scripts в Project.
2. Создать MonoBehaviour Script с именем LevelManager.
3. Открыть LevelManager.cs в текстовом редакторе.
4. Заменить весь текст полным листингом из раздела «Код · LevelManager.cs», соединив его части по порядку.
5. Сохранить файл.
6. Повторить создание файла и вставку полного листинга для каждого имени из следующего списка.
7. Вернуться в Unity после сохранения всех файлов.
8. Открыть Window → General → Console.

3

Добавить каждый файл

LevelManager.cs, SaveService.cs, GameManager.cs, MenuController.cs, PauseController.cs, WorldBuilder.cs, HoldButton.cs, PlayerController.cs, Collectible.cs, Hazard.cs, ExitZone.cs, VisualTheme.cs, AudioFeedback.cs, CollectibleVisual.cs, GameBootstrap.cs. SaveService — статический служебный класс; его полный листинг заменяет шаблон, к объекту он не прикрепляется.

4

Проверка

Все 15 файлов добавлены, имена совпадают с листингами, компиляция завершена без красных ошибок. До установки всего набора могут появляться сообщения о ещё не созданных классах.

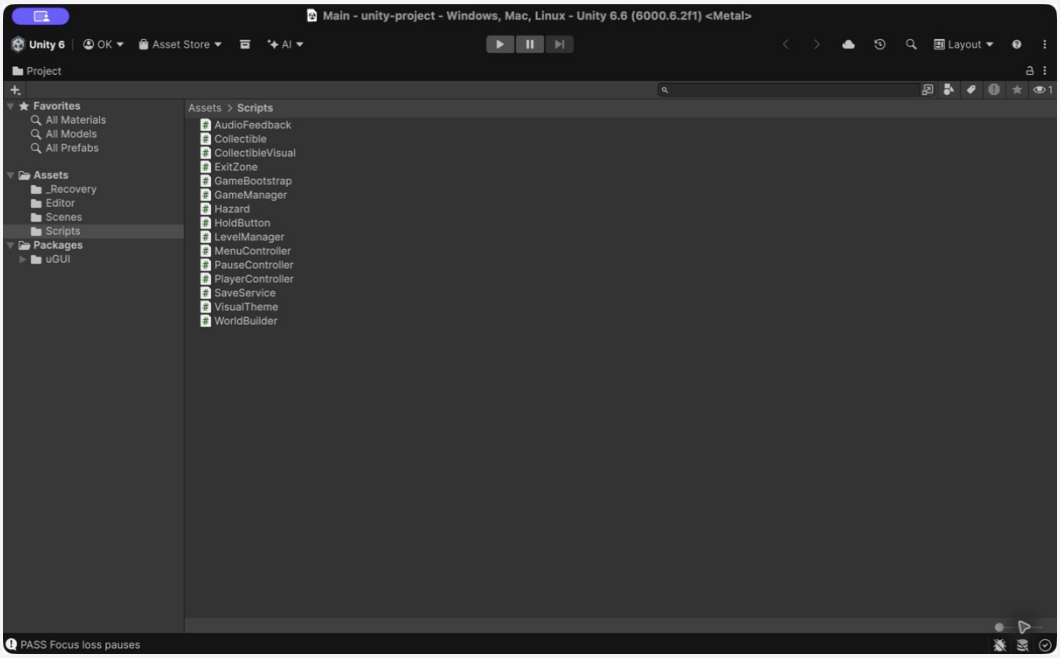
5

Если результат отличается

Не вставлять заголовки и номера строк. Не оставлять исходный шаблон рядом с новым классом. Если класс уже существует, заменить содержимое его файла; не создавать дубликат.

6

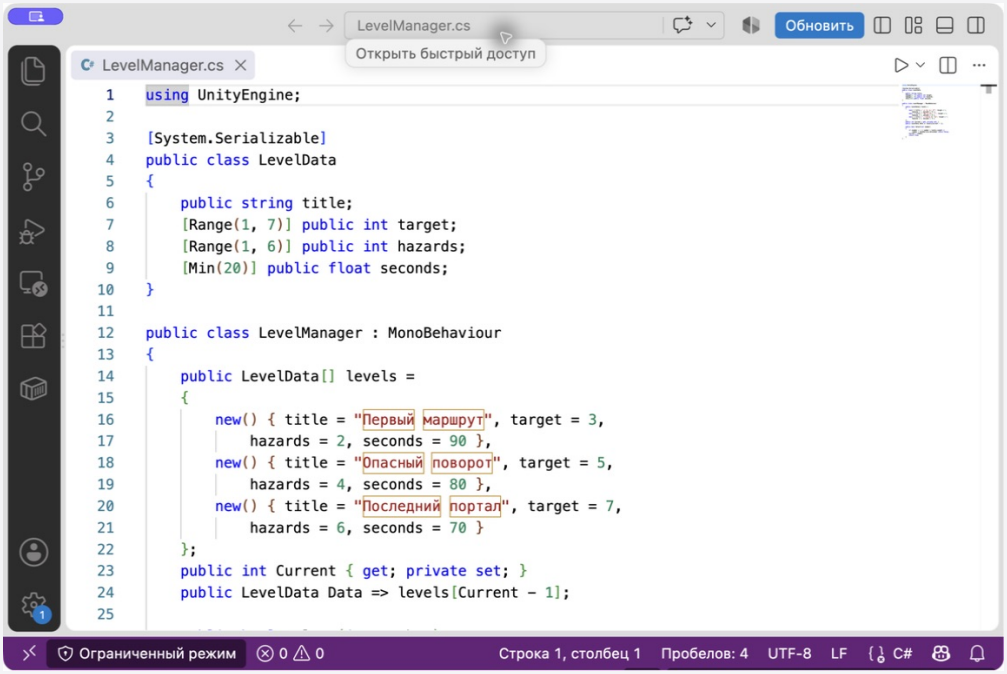
Файлы скриптов в Unity



В окне Project открыта папка Assets → Scripts. Найти LevelManager; если файл уже есть, открыть его, не создавая дубликат. Папка _Recovery на снимке служебная и в комплект сдачи не входит.

7

Открытый файл LevelManager.cs



LevelManager.cs открыт в редакторе кода. Имя класса LevelManager совпадает с именем файла. Здесь видны данные трёх уровней; полный текст для вставки приведён в разделе «Код · LevelManager.cs».

ПП ПМ.12 · Раздел 11

05. Собрать GameRoot в сцене

1

Что это и зачем

GameRoot — единственный постоянный объект контрольной сцены. LevelManager хранит данные уровней, WorldBuilder строит поле, GameManager управляет состоянием игры.

2

Действия

1. Открыть Assets/Scenes/Main.unity.
2. Выбрать GameObject → Create Empty.
3. Переименовать объект в GameRoot.
4. Нажать Add Component в Inspector.
5. Добавить LevelManager.
6. Добавить WorldBuilder.
7. Добавить GameManager.
8. Сохранить сцену.

3

Проверка

GameRoot содержит Transform и три компонента. Сравнить Inspector со снимком. MenuController, PauseController и AudioFeedback вручную добавлять не нужно: их добавляет GameManager.Awake.

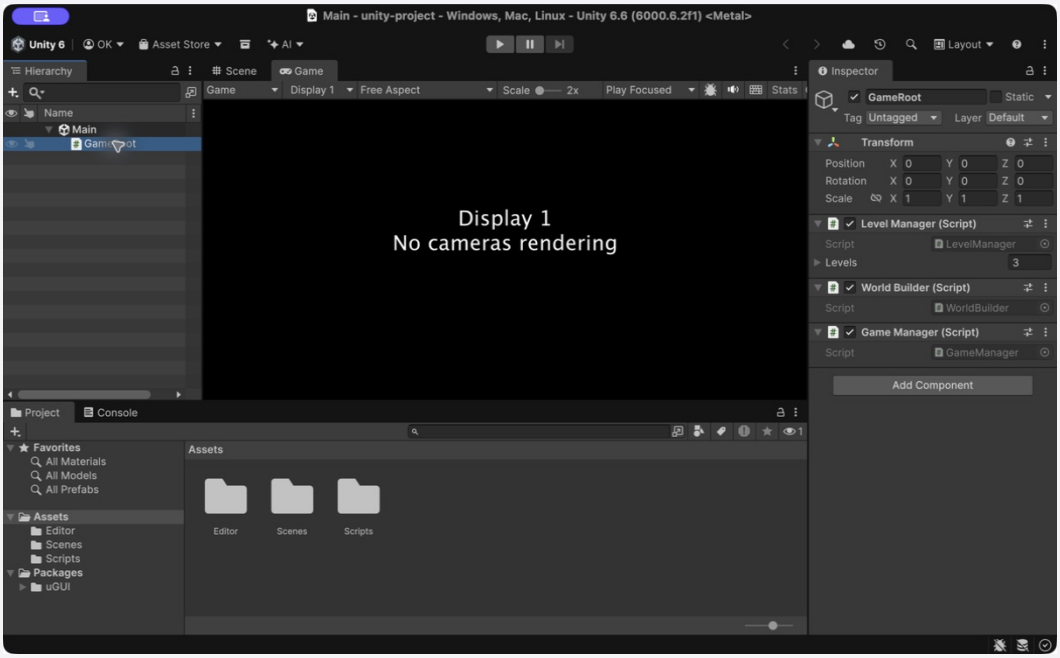
4

Если результат отличается

Если класс не найден, исправить Console. Нужен один GameRoot. Без него GameBootstrap создаст CrystalQuest с настройками по умолчанию. До Play камеры нет.

5

Настройка в Unity



GameRoot содержит Level Manager, World Builder и Game Manager. Play выключен; надпись No cameras rendering ожидаема: камера и интерфейс создаются при запуске.

ПП ПМ.12 · Раздел 12

06. Настроить три уровня

1

Что это и зачем

Массив — упорядоченный набор данных. В levels хранятся три LevelData. Индекс массива начинается с 0; номер для игрока — с 1.

2

Действия

1. Остановить Play.
2. Выбрать GameRoot.
3. Развернуть Level Manager → Levels.
4. Проверить число записей справа от Levels = 3.
5. В первой записи «Первый маршрут» задать Target = 3, Hazards = 2, Seconds = 90.
6. В второй записи «Опасный поворот» задать Target = 5, Hazards = 4, Seconds = 80.
7. В третьей записи «Последний портал» задать Target = 7, Hazards = 6, Seconds = 70.
8. Сохранить сцену.

3

Проверка

В первом уровне 3 кристалла и 2 опасности; далее добавляются объекты и стены, время уменьшается.

4

Если результат отличается

В контрольном поле предусмотрено максимум 7 кристаллов и 6 опасностей. Чтобы увеличить предел, добавить позиции в WorldBuilder и изменить ограничения LevelData.

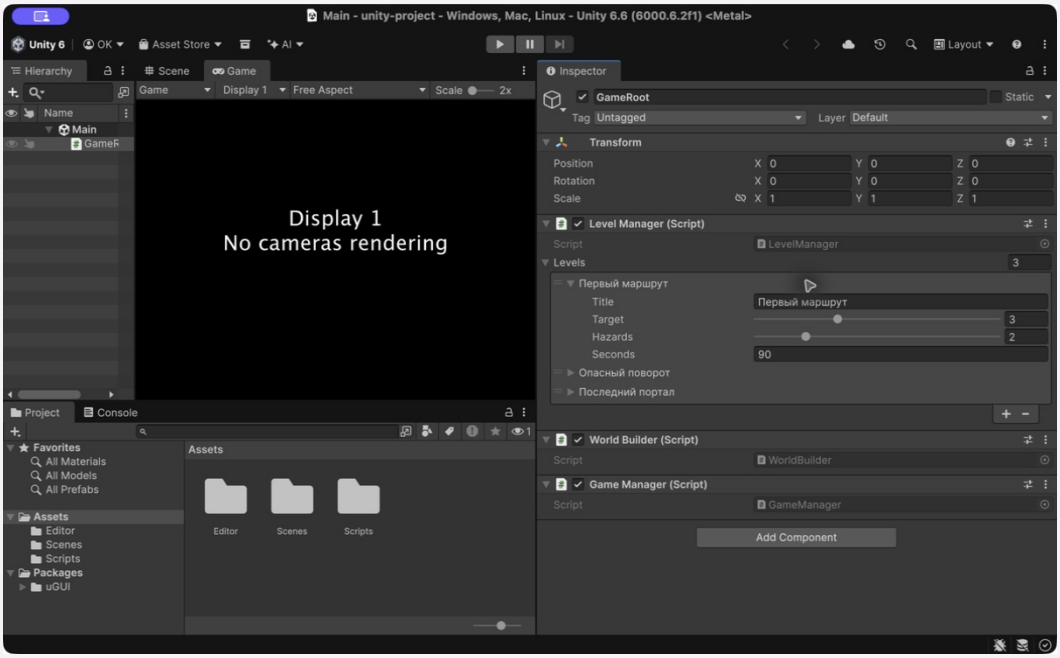
5

Адаптировать свой вариант

Заменить Target параметрами своей механики: длиной порядка, числом доставок, вместимостью или составом набора. Условия смотреть в своей карточке.

6

Настройка в Unity



В Inspector раскрыт Level Manager → Levels: три записи. Для первого уровня Target = 3, Hazards = 2, Seconds = 90. Параметры меняются вне Play.

ПП ПМ.12 · Раздел 13

07. Получить главное меню первым запуском

1

Что это и зачем

Awake выполняется при создании компонента, Start — перед его первым обновлением. GameManager.Awake добавляет управляющие компоненты и вызывает WorldBuilder.BuildCamera. Затем MenuController.Start создаёт Canvas, EventSystem и панели.

2

Действия

1. Убедиться, что Console не содержит красных ошибок.
2. Нажать Play.
3. Открыть вкладку Game.
4. Найти меню «Кристальный маршрут».
5. Найти в Hierarchy Main Camera.
6. Найти Canvas.
7. Раскрыть GameRoot в Inspector и найти добавленные компоненты.

3

Проверка

Меню впервые появилось из добавленного кода. На новом сохранении доступен уровень 1. Снимок ниже показывает сохранённый запуск контрольного примера.

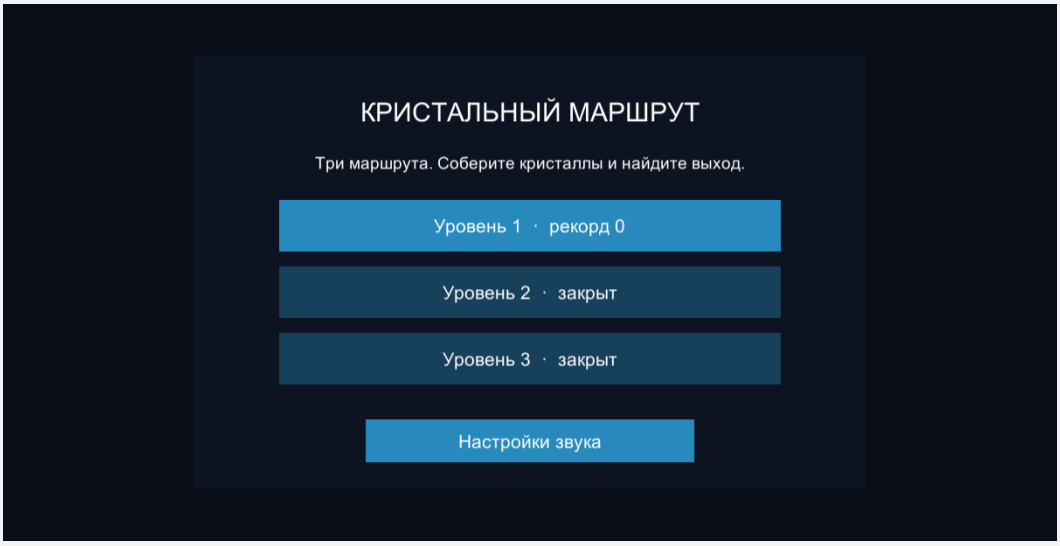
4

Если результат отличается

Если уровни уже открыты, в PlayerPrefs сохранились прежние данные примера. Для чистого учебного теста сменить префикс в SaveService в своей копии; не удалять все PlayerPrefs.

5

Снимок результата



Главное меню контрольного примера. Закрытые уровни не должны запускаться.

ПП ПМ.12 · Раздел 14

08. Получить поле из данных уровня

1

Что это и зачем

Кнопка Level1 вызывает GameManager.StartLevel(1). Он проверяет доступ через LevelManager.Select, сбрасывает счётчики и вызывает WorldBuilder.Build. Этот метод создаёт LevelObjects, стены, Player, кристаллы, опасности и Exit.

2

Действия

1. Нажать кнопку уровня 1.
2. Раскрыть LevelObjects в Hierarchy.
3. Выбрать Player.
4. Найти Rigidbody2D, CircleCollider2D и PlayerController в Inspector.
5. Выбрать Crystal1.
6. Найти PolygonCollider2D и Collectible.
7. Удерживать экранную стрелку в Game.
8. Отпустить стрелку.

3

Проверка

Вместо меню показано поле: игрок, три кристалла, две опасности и выход. HoldButton передаёт направление в GameManager, PlayerController двигает игрока. После отпускания движение прекращается.

4

Если результат отличается

Не расставлять второй набор объектов вручную: WorldBuilder уже создаёт их при старте. Если поле не появилось, проверить Console и наличие трёх компонентов GameRoot.

ПП ПМ.12 · Раздел 15

09. Разделите постоянные и временные объекты

1

Что это и зачем

Объекты сцены сохраняются на диске. Объекты, созданные скриптом во время Play, существуют только в текущем запуске. В примере WorldBuilder создаёт игровое поле, MenuController создаёт Canvas.

2

Действия

1. Остановить Play.
2. Выбрать GameRoot в Hierarchy.
3. Запомнить постоянные компоненты в Inspector.
4. Нажать Play.
5. Запустить уровень 1.
6. Раскрыть LevelObjects в Hierarchy.
7. Найти Player, Crystal1 и Exit.
8. Остановить Play.

3

Проверка

LevelObjects и Canvas исчезли, GameRoot остался. Изменения параметров Levels вне Play сохраняются.

4

Если результат отличается

Не размещать постоянные настройки только в объектах, созданных во время игры. Координаты примера менять в WorldBuilder.cs; параметры уровней — в Level Manager вне Play.

ПП ПМ.12 · Раздел 16

Перенос общих функций в свой вариант

1

Сначала составить карту соответствий

В отчёте записать: данные уровня → собственный класс данных; StartLevel → сброс и создание объектов; TryFinishLevel → проверка особого правила; Finish → единственное завершение; ToMenu → очистка; MenuController → существующий Canvas. Поиск класса: в редакторе кода открыть поиск по проекту и найти обработчик собственного выхода, затем место изменения счётчика. Менять найденные обработчики, не добавлять конкурирующий GameManager.

2

Уровни и создание объектов

Скопировать структуру LevelData и массив levels из LevelManager в собственный класс. Вместо target добавить поля нужных норм, вместимости, адресатов, последовательностей или связей. В StartLevel сначала проверить номер и доступ, затем восстановить временные переменные, удалить старый корень объектов и создать новый из выбранной записи. Объекты должны иметь уникальный ID в пределах уровня. Для норм больше 7 добавить столько достижимых позиций, сколько задано карточкой: массив образца ограничен семью кристаллами, он не создаёт автоматически 9 или 20 предметов.

3

Условие завершения

В обработчике выхода заменить только проверку Collected < target на проверку особого правила по соответствующему разделу «Адаптация». При невыполнении вывести причину, оставить Playing и не вызывать Complete. При выполнении вычислить результат, один раз вызвать SaveService.Complete, затем Finish(true). Во всех обработчиках сначала отклонять события вне Playing. Нажатие после Won не должно повторно начислять результат.

4

Сохранения и кнопки

Скопировать SaveService.cs, изменить Prefix на PP12.VariantNN.v1. и использовать один класс сохранений во всех вызовах. Существующие кнопки направить на свои StartLevel, TogglePause, ToMenu и повтор StartLevel(current). Для UnityEvent: вне Play выбрать кнопку → Inspector → Button → On Click → «+» → перетащить объект менеджера → выбрать его публичный метод; для старта выбрать метод с int и ввести номер уровня. Старый обработчик удалить только после проверки нового. Для созданных кодом кнопок использовать onClick.AddListener как в MenuController.

5

Проверка и восстановление

Сначала проверить первый уровень; затем повтор, паузу и меню. Если исчезла собственная механика, вернуть последнюю рабочую копию обработчика и переносить по одной функции. Если одно нажатие вызывает два перехода, проверить дубли подписок в On Click и AddListener. Сохранять следующий коммит только после повторения исходного случая.

ПП ПМ.12 · Раздел 17

11. Проверить рост сложности

1

Что это и зачем

Сложность должна изменять действия игрока. В примере усложняются число целей, опасностей, планировка и лимит времени.

2

Действия

1. Запустить уровень 1.
2. Собрать каждый жёлтый ромб.
3. Войти в зелёный портал.
4. Нажать «Следующий уровень».
5. Сравнить число объектов и стен.
6. Пройти второй уровень.
7. Открыть третий уровень.
8. Записать различия в таблицу отчёта.

3

Проверка

Три уровня проходимы и требуют разных маршрутов.

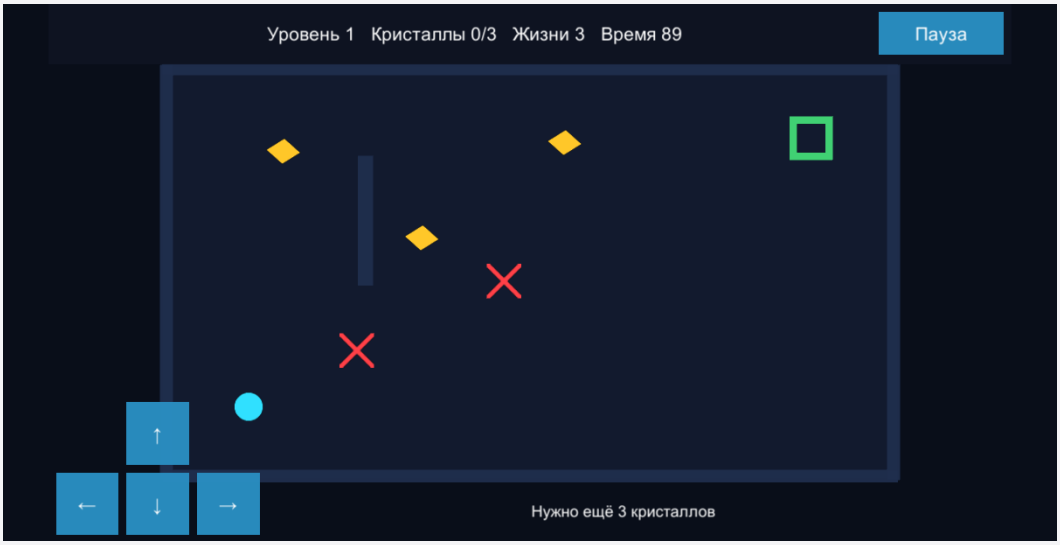
4

Если результат отличается

Если цель недостижима, проверить позиции объектов и Collider 2D стен. Предмет внутри стены нельзя считать корректным усложнением.

5

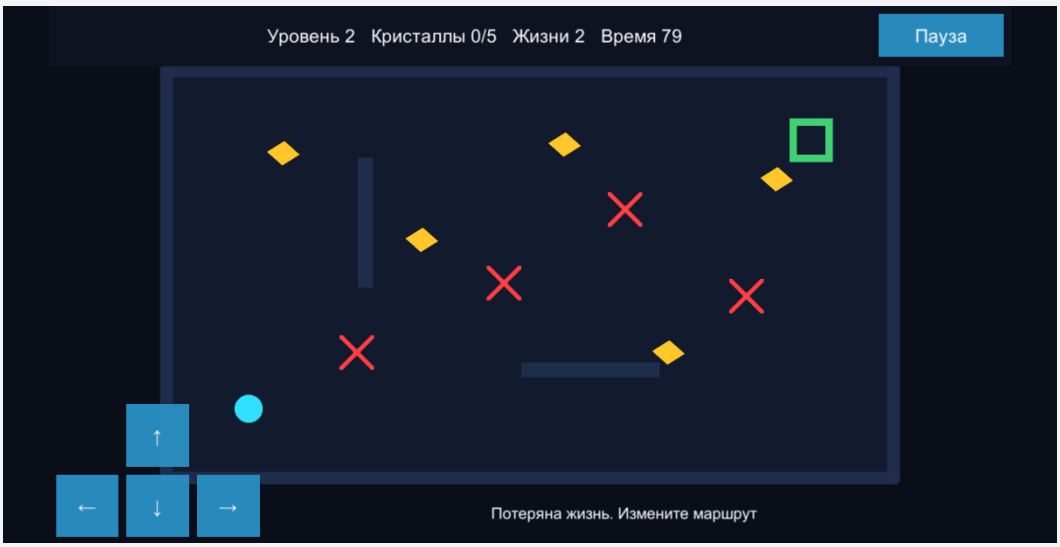
Состояние игры



Первый уровень: 3 кристалла, 2 опасности, 90 секунд.

6

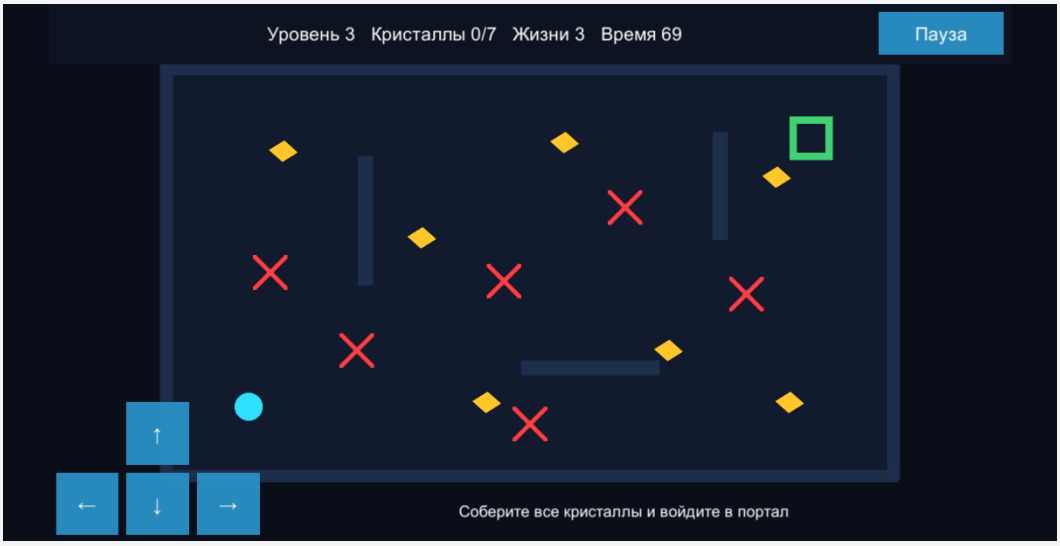
Состояние игры



Второй уровень: 5 кристаллов, 4 опасности и дополнительная стена.

7

Снимок результата



Третий уровень: 7 кристаллов, 6 опасностей и дополнительные стены.

ПП ПМ.12 · Раздел 18

12. Подключить прогрессию

1

Что это и зачем

Победа → SaveService.Complete → сохранённый Unlocked → доступная кнопка. LevelManager.Select повторно проверяет номер, поэтому закрытый уровень не запустится даже прямым вызовом.

2

Действия

1. Открыть LevelManager.cs.
2. Найти метод Select.
3. Найти сравнение number > SaveService.Unlocked.
4. Запустить игру с новым сохранением.
5. Попробовать выбрать уровень 2.
6. Пройти уровень 1.
7. Вернуться в меню.
8. Нажать уровень 2.

3

Проверка

До победы уровень 2 не запускается; после победы становится доступен.

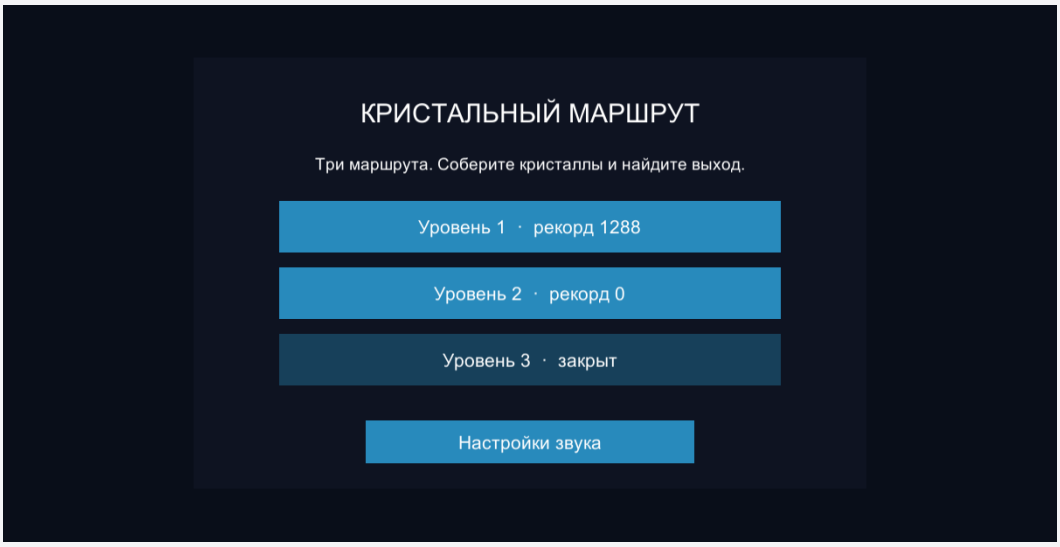
4

Если результат отличается

Нельзя открывать уровень при входе в него или при поражении. SaveService.Complete вызывается только после проверки победы.

5

Снимок результата



После победы доступен следующий уровень; рекорд первого виден в меню.

ПП ПМ.12 · Раздел 19

13. Проверить экран результата

1

Что это и зачем

Состояние — одно из значений Menu, Playing, Paused, Won, Lost. GameManager хранит состояние, а MenuController включает соответствующую панель.

2

Действия

1. Запустить уровень.
2. Войти в портал до сбора цели.
3. Прочитать подсказку.
4. Собрать цель и снова войти в портал.
5. На экране победы нажать «Повторить уровень».
6. Проверить восстановление жизней и предметов.
7. Довести жизни до нуля.
8. На экране поражения нажать «В меню».

3

Проверка

Ранний выход не завершает уровень. Повтор начинается с чистого состояния, поражение не открывает контент.

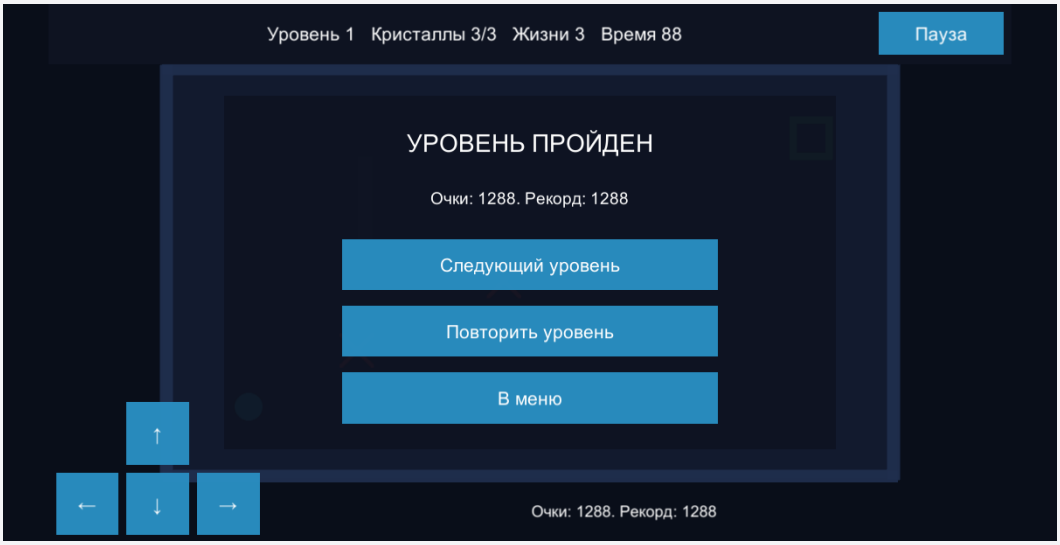
4

Если результат отличается

Для особого правила вместо одного числа Collected проверять весь порядок, доставки или набор. Не засчитывать поднятый груз как доставленный.

5

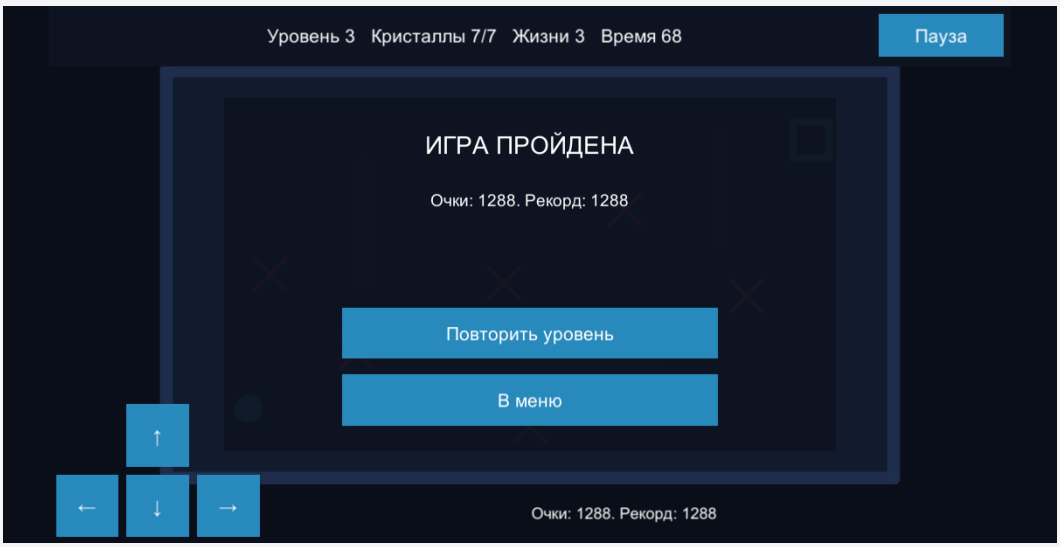
Снимок результата



Победа показывает результат и кнопку перехода к следующему уровню.

6

Состояние игры



После третьего уровня игра завершена; кнопки следующего уровня нет.

14. Включить паузу

1

Что это и зачем

Time.timeScale = 0 останавливает игровое время и физику. UI продолжает обрабатывать нажатия. Состояние Paused блокирует сбор и повреждения.

2

Действия

1. Начать уровень.
2. Запомнить оставшееся время.
3. Нажать «Пауза».
4. Подождать несколько секунд.
5. Убедиться, что таймер не изменился.
6. Нажать экранную стрелку.
7. Нажать «Продолжить».
8. Проверить движение и уменьшение времени.

3

Проверка

На паузе персонаж и таймер стоят; после «Продолжить» игра работает.

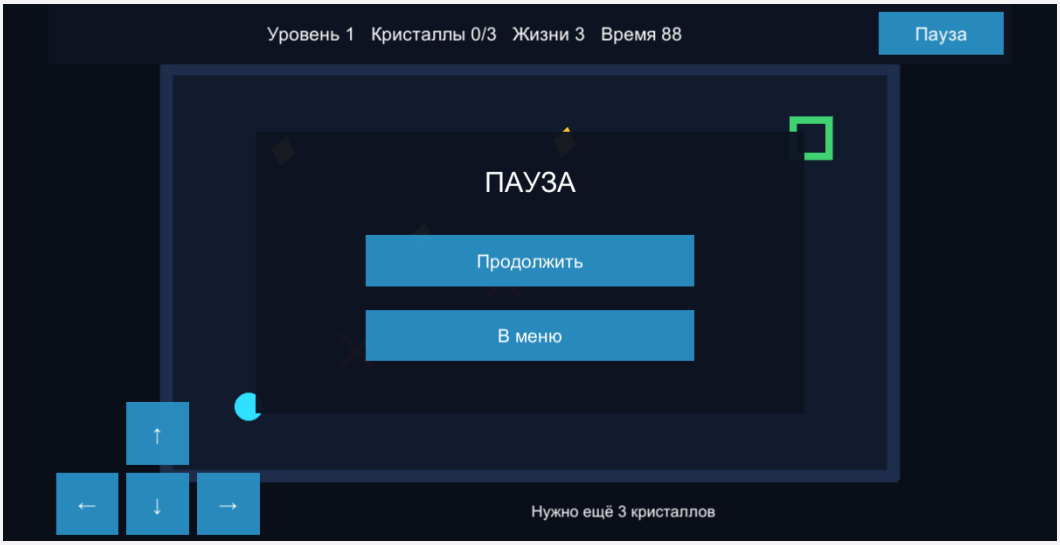
4

Если результат отличается

Если UI зависает, не отключать EventSystem. Если продолжается движение, проверить IsPlaying в PlayerController и сброс MobileDirection.

5

Снимок результата



Панель паузы содержит «Продолжить» и «В меню».

ПП ПМ.12 · Раздел 21

15. Проверить сворачивание

1

Что это и зачем

OnApplicationFocus сообщает о потере фокуса окна. OnApplicationPause сообщает о приостановке приложения. Оба события вызывают Pause, но возвращение фокуса само игру не возобновляет.

2

Действия

1. Запустить уровень.
2. Переключиться на другое приложение.
3. Вернуться в окно Game.
4. Убедиться, что видна пауза.
5. Подождать и проверить таймер.
6. Нажать «Продолжить».

3

Проверка

Возврат требует осознанного действия игрока.

4

Если результат отличается

В редакторе поведение фокуса зависит от режима Game View. Проверку Android Home/возврат отмечать отдельно и только после фактического запуска сборки на устройстве.

5

Если редактор перехватывает фокус

В Game View выбрать режим Play Unfocused, затем снова выполнить Alt+Tab и возврат. Если событие по-прежнему не воспроизводится в редакторе, записать ограничение фактической проверки. При доступной настольной сборке отдельно повторить там; её создание не становится обязательным условием. Сам callback OnApplicationFocus(false) должен вызывать Pause, а true не должен вызывать TogglePause.

ПП ПМ.12 · Раздел 22

16. Сохранить настройки

1

Что это и зачем

PlayerPrefs хранит небольшие значения между запусками. Ключ — уникальное имя записи. Префикс PP12.Crystal.v1. отделяет данные примера от других проектов.

2

Действия

1. Открыть главное меню.
2. Нажать «Настройки звука».
3. Нажать «−» несколько раз до 40%.
4. Нажать «Назад».
5. Остановить Play.
6. Нажать Play снова.
7. Открыть настройки.

3

Проверка

Громкость осталась 40%. Вне Play изменение громкости не требуется.

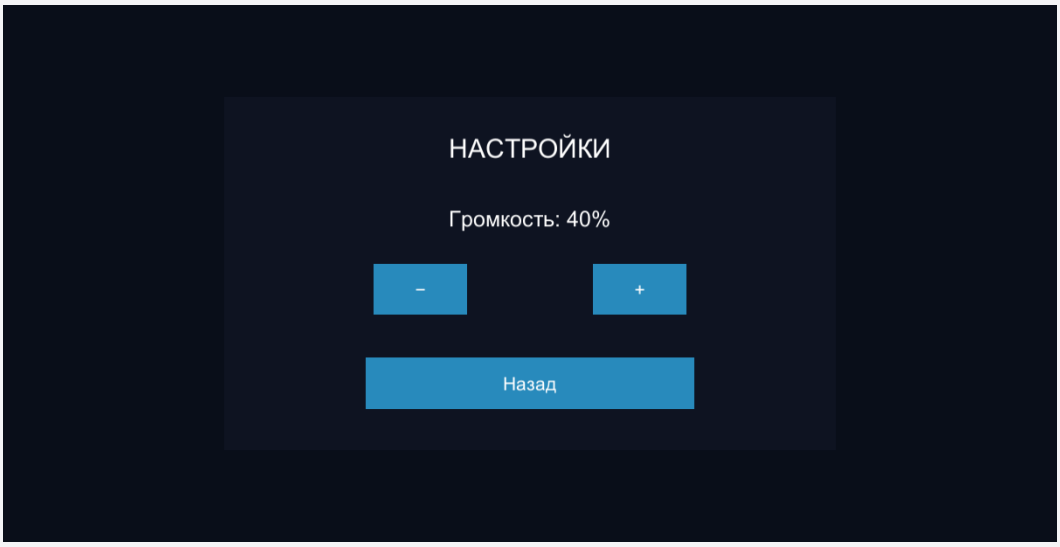
4

Если результат отличается

SaveService.SetVolume одновременно обновляет AudioListener.volume, значение PlayerPrefs и вызывает Save. Для своего варианта задать уникальный префикс.

5

Снимок результата



Экран настроек показывает сохранённую громкость.

ПП ПМ.12 · Раздел 23

17. Проверить сохранение прогресса

1

Что это и зачем

Постоянные данные: открытые уровни, рекорд и громкость. Временные: жизни, оставшееся время, собранные предметы, груз, текущий порядок. В этом примере попытка не восстанавливается посередине.

2

Действия

1. Пройти уровень 1.
2. Записать рекорд.
3. Остановить Play.
4. Нажать Play повторно.
5. Проверить доступность уровня 2.
6. Пройти уровень 1 с меньшим количеством очков.
7. Проверить, что старый рекорд не уменьшился.

3

Проверка

Прогресс сохранился, а худший результат не заменил лучший.

4

Если результат отличается

Не вызывать DeleteAll при старте. Для чистого теста использовать копию проекта и новый префикс; это не удаляет чужие сохранения. Не путать PlayerPrefs редактора и отдельной сборки.

5

Сопоставление рекордов

Для каждой области заранее записать формулу: успешные доставки/действия дают основную часть, время или оставшийся ресурс — различие при равенстве. Вариантная цель и штраф входят в исходные данные. Complete вызывается только из успешного выхода; повторное прохождение худшим результатом не должно менять Best(level). Для сравнения выбрать одинаковый уровень, уменьшить время или ресурс перед вторым успешным выходом.

ПП ПМ.12 · Раздел 24

18. Оформить тестовый случай

1

Что это и зачем

Тестовый случай — конкретные исходные условия, действие и ожидаемый результат. Фактический результат заполняется после запуска.

2

Действия

1. Открыть итоговую таблицу раздела 3 «Отчет о выполнении.docx».
2. В первом столбце дополнить ID случая исходными условиями и последовательностью действий; во втором записать ожидаемый результат, в третьем — фактический. В четвёртом указать статус и ссылку на собственный рисунок или журнал.
3. Заполнить T01–T16 и добавить строки для особых проверок своей карточки. Для нескольких наблюдений увеличить строку или перенести подробное описание в раздел соответствующего этапа.
4. Выполнить каждую последовательность. Реальное наблюдение и статус записать после выполнения.
5. Приложить доказательство и проверить, что по записи можно повторить случай.

3

Проверка

Для каждого теста понятны исходные данные и способ повторения.

4

Если результат отличается

Не ставить «пройдено» заранее. Если проверка не выполнена, так и напишите. Автопроверка методов не заменяет проверку реальных кнопок и столкновений.

5

Подготовка T13–T16

T13–T14: взять особую проверку своей карточки, записать перед действием счётчики/флаги, ошибочный ввод, ожидаемое сохранение либо изменение; повторить на уровнях 1,2,3. T15: в списке размеров Game View выбрать «+» → Fixed Resolution, создать 1280×720 и 1920×1080; поочерёдно включить и проверить подписи и доступность всех кнопок в каждом состоянии. При обрезании собственного Canvas задать Canvas Scaler → Scale With Screen Size, Reference Resolution 1280×720, Match 0.5; привязать края HUD и кнопок RectTransform Anchors к соответствующим краям, перепроверить оба размера. T16: вне Play запомнить число постоянных объектов, затем пять раз открыть уровень и вернуться в меню. Во время игры должен существовать ровно один активный корень уровня; в меню ни одного. При росте найти место создания и добавить отключение/Destroy старого корня перед новым; дождаться следующего кадра перед подсчётом Destroy. Console не должна накапливать сообщения.

ПП ПМ.12 · Раздел 25

19. Исправить и перепроверьте ошибку

1

Что это и зачем

Ошибка описывается через наблюдаемый симптом. Исправление подтверждается повторением исходного теста и соседних переходов.

2

Действия

1. Открыть Window → General → Console.
2. Выбрать красное сообщение, если оно есть.
3. Открыть указанную строку двойным щелчком.
4. Записать шаги, вызвавшие ошибку.
5. Исправить причину.
6. Повторить те же шаги.
7. Проверить повтор уровня и возврат в меню.

3

Проверка

Ошибка не воспроизводится, остальные переходы работают.

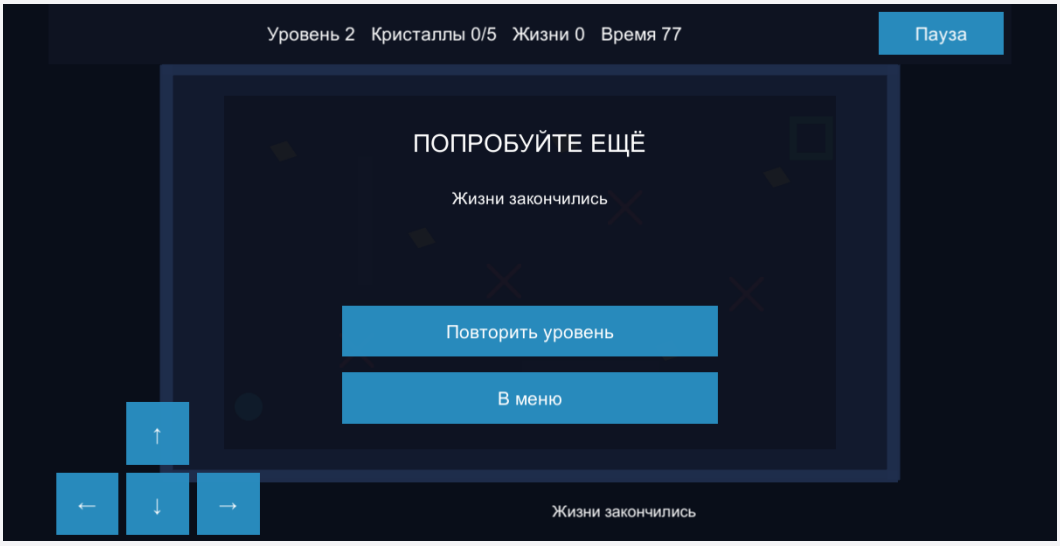
4

Если результат отличается

Очистка Console не исправляет причину. В журнале указать: симптом, причина, изменённый файл, повторная проверка. Не придумывать найденные ошибки.

5

Снимок результата



Экран поражения: доступен повтор и возврат в меню, переход вперёд отсутствует.

ПП ПМ.12 · Раздел 26

20. Наблюдать производительность

1

Что это и зачем

Profiler показывает затраты времени и памяти. Результаты редактора зависят от компьютера; фиксировать условия и не переносите их автоматически на телефон.

2

Действия

1. Открыть Window → Analysis → Profiler; в поле Target выбрать Play Mode. Включить Record и модуль CPU Usage.
2. Открыть уровень 3, воспроизвести один и тот же маршрут 30 секунд; первые кадры создания объектов анализировать отдельно. Остановить Record.
3. Выбрать устойчивый участок CPU Usage, переключить нижнюю панель в Hierarchy, записать Total ms, GC Alloc и число выбранных кадров; для Memory записать используемую память при тех же условиях. Сохранить снимок «до».
4. Найти в своём коде обоснованную причину лишней работы. Например, если GetComponent выполняется в Update, сохранить ссылку один раз в Awake и использовать поле. Если ссылки уже сохранены, не добавлять искусственную ошибку: выбрать реальное повторное создание ресурса или лишний поиск.
5. Сохранить изменение, дождаться компиляции и повторить тот же маршрут, размер Game View и целевой режим. Записать показатели и снимок «после».
6. Сравнить наблюдения, указать выбранный эпизод и ограничения. Отсутствие измеримого улучшения — допустимый фактический результат; отменить изменение, если оно ухудшает работу.

3

Проверка

После переходов старые LevelObjects удаляются. Спрайты создаются один раз на форму и повторно используются из словаря.

4

Если результат отличается

Первый кадр включает создание объектов; сравнивать сходные участки игры. Если память растёт постоянно, проверить Clear и уничтожение созданных текстур.

5

Адаптировать свой вариант

Выбрать одно обоснованное улучшение своего проекта: повторное использование ресурсов, удаление старых объектов или устранение лишнего поиска каждый кадр. Измерить до и после.

ПП ПМ.12 · Раздел 27

21. Снять доказательства

1

Что это и зачем

Статичный снимок должен показывать состояние и значимые числа. Он сопровождает текст, а не заменяет объяснение.

2

Действия

1. Открыть нужное состояние игры.
2. Убедиться, что текст и кнопки целиком помещаются в Game View.
3. Сохранить снимок средствами операционной системы.
4. Назвать файл 01_menu.png или другим последовательным именем.
5. Вставить снимок в соответствующий раздел отчёта.
6. Добавить подпись и одно предложение о проверяемом результате.

3

Проверка

Снимки крупные, читаемые, без лишних окон и личных данных.

4

Если результат отличается

Не выдавать схему или старый снимок за новый результат. Для сравнения «до/после» использовать одинаковый размер окна.

ПП ПМ.12 · Раздел 28

22. Опубликовать Unity-проект в репозитории

1

Что это и зачем

Репозиторий хранит исходники и историю изменений. Коммит — сохранённая версия; его хеш позволяет открыть именно сданный результат. Преподавателю нужна ссылка на проект, который можно клонировать и запустить.

2

Действия

1. Остановить Play, сохранить сцену и закрыть Unity. В папке проекта создать .gitignore с содержимым следующего блока.
2. Создать README: номер и название варианта, версия Unity, путь к сцене, управление, правило победы и поражения, порядок запуска.
3. На GitHub открыть «+» → New repository; задать собственное имя, не добавлять удалённые README и .gitignore при первом подключении существующей локальной папки. Создать репозиторий; скопировать HTTPS URL через Code.
4. Открыть терминал именно в папке с Assets, Packages и ProjectSettings. Выполнить git status. Если это ещё не репозиторий, выполнить git init -b main. Если уже репозиторий, сохранить имеющуюся историю и ветку.
5. Выполнить git config user.name и git config user.email; при отсутствии задать собственные данные. Выполнить git add Assets Packages ProjectSettings .gitignore README.md, затем git diff --cached --stat. Проверить отсутствие служебных папок.
6. Выполнить git commit -m "Завершить игру ПМ.12". Выполнить git remote -v. Если origin отсутствует: git remote add origin СВОЙ_HTTPS_URL. Если origin уже указывает на собственный проект, оставить его. При чужом origin использовать новую папку собственного репозитория и перенести только три папки Unity и документы.
7. Узнать ветку: git branch --show-current. Выполнить git push -u origin ИМЯ_СВОЕЙ_ВЕТКИ. Войти в собственную учётную запись предложенным Git способом. При отклонении push не использовать force: проверить адрес и историю, сохранить локальный коммит.
8. На GitHub обновить страницу; открыть Assets, Packages, ProjectSettings и README. Для закрытого репозитория Settings → Collaborators → Add people; дать доступ назначенному проверяющему.

3

Исключения .gitignore

```
Library/
Temp/
Logs/
obj/
UserSettings/
Build/
Builds/
.vs/
.DS_Store
*.csproj
*.sln
```

4

Проверка

На странице репозитория видны три папки Unity и README; служебные каталоги отсутствуют.

5

Если результат отличается

Ссылка только на сцену, отдельный файл или пустой репозиторий не подходит. Для закрытого репозитория предоставить преподавателю доступ.

23. Проверить ссылку и сдать результат

1

Что это и зачем

Проверка новой копии выявляет файлы, оставшиеся только на собственном компьютере. ZIP для сдачи больше не требуется: обязательна ссылка на репозиторий.

2

Действия

1. Скопировать HTTPS URL собственного репозитория через Code. В другой, пустой родительской папке выполнить git clone СВОЙ_HTTPS_URL PP12_Check.
2. В Unity Hub выбрать Add → Add project from disk → PP12_Check. Открыть версией из README, дождаться импорта. Открыть сцену по указанному пути.
3. Включить Play. Проверить запуск, победу, повтор и сохранение по T01–T16. Исправления отправить в исходном проекте новым коммитом и получить их в контрольной копии командой git pull --ff-only.
4. После успешной проверки выполнить в контрольной копии git rev-parse HEAD. Скопировать хеш и URL в раздел 3 «Отчет о выполнении.docx».
5. В Word заполнить поля, заменить рамки своими рисунками через Вставка → Рисунки → Это устройство. Обновить поля Ctrl+A, F9; проверить номера рисунков и сохранить.
6. Передать DOCX, URL, хеш и установленные формы через действующий канал сдачи. При дополнительном PDF сохранить его из Word; DOCX остаётся основной редактируемой формой.

3

Проверка

Проект запускается из клонированной копии, преподаватель имеет доступ к ссылке, хеш соответствует проверенной версии.

4

Если результат отличается

Если в новой копии потеряны ресурсы, проверить Assets и .meta. Исправить репозиторий, создать новый коммит и повторить проверку.

ПП ПМ.12 · Раздел 30

24. Образец оформления результатов

1

Описание уровня

Уровень 1: цель 3 кристалла, 2 опасности, 90 секунд, одна внутренняя стена. Уровень 2: 5, 4, 80 секунд, две стены. Уровень 3: 7, 6, 70 секунд, три стены. Для своего варианта заменить эти данные.

2

Строка проверки · пример формата

T09 | уровень 1, время 70 | нажать «Пауза», ждать 5 секунд | ожидается: время 70, движения нет | фактически: заполнить после проверки | статус и рисунок: заполнить. Это пример строки, а не выполненная проверка.

3

Запись об исправлении · пример формата

Симптом: после возврата из паузы не движется игрок. Причина: Time.timeScale не возвращён в 1. Изменение: ветка продолжения в TogglePause. Подтверждение: повтор T09 и T10. Это пример записи, не утверждение об ошибке в собственном проекте.

4

Вывод

Указать разработанные функции, реально выполненные проверки и ограничения в итоговой проверке отчёта. Результат Play в редакторе не обозначать как проверку Android. Отдельные заключение и список источников в форму не добавляются.

ПП ПМ.12 · Раздел 31

Адаптация · Выборочный сбор

1

Что хранить

Хранить тип предмета отдельно от внешнего вида. Данные уровня задают требуемое число правильных предметов и количество помех.

2

Как изменить логику

В обработчике контакта сначала проверить тип. Если тип подходит, увеличить правильный счёт и отключить предмет. Иначе применить штраф исходного варианта. Победу проверять по правильному счёту, а не по всем контактам.

3

Место изменения и порядок обработки

В обработчике контакта собственного предмета хранить itemId и тип, а использованные ID — в HashSet<int>. Сначала проверить Playing и наличие ID в наборе. Неподходящий тип не прибавляет progress: применить штраф из карточки, обновить ресурс и проверить поражение. Правильный тип записать в used, увеличить progress на 1, отключить только этот объект. Для варианта 04 счёт относится к собранному заказу: победа лишь при полном составе в зоне доставки. При повторе очистить used и восстановить объекты. В HUD отдельно вывести progress/норму и жизни, заряд, ошибки или время. Для нормы 9 создать девять разных ID и позиций; клонирование одного ID даст неверную проверку.

4

Уровни и сброс

В начале уровня обнулите ошибки и правильный счёт. Между запусками хранить только лучший результат и открытый уровень.

5

Проверка границы

Коснуться неподходящего предмета перед последним правильным: победа не должна наступать преждевременно.

ПП ПМ.12 · Раздел 32

Адаптация · Порядок действий

1

Что хранить

Массив хранит ожидаемый порядок. Переменная nextIndex указывает следующую ожидаемую позицию; в начале она равна 0.

2

Как изменить логику

Сравнить идентификатор предмета с order[nextIndex]. При совпадении увеличить nextIndex. Перед следующим обращением к массиву проверить nextIndex < order.Length. Когда индекс равен длине, цепочка завершена.

3

Место изменения и порядок обработки

В данных уровня хранить order — массив идентификаторов, в попытке nextIndex = 0. В контакте сначала проверить Playing и nextIndex < order.Length; при завершённой цепочке новый контакт ничего не делает. Затем сравнить ID с order[nextIndex]. Совпадение: увеличить индекс, показать следующий ожидаемый ID. Ошибка: применить правило карточки и проверить порог поражения; для рун вернуть nextIndex в 0 и снова включить все руны этой попытки. Для нот неверный объект не отключать. Для станций повтор уже пройденного ID не продвигает индекс. Вариант 10: после «Положить» увеличить delivered; при delivered меньше нормы следующая команда «Взять», иначе «Отчёт»; «Нести» допустима лишь с грузом. Проверить ошибку до первого шага, в середине и контакт после последнего. HUD показывает ожидаемое действие и ошибки.

4

Уровни и сброс

Сброс последовательности должен восстановить доступность уже использованных объектов, иначе повторить её будет невозможно. Политику штрафа сохранить из своего варианта.

5

Проверка границы

После последнего элемента повторный контакт не вызывает выход за границы массива. Новая попытка начинается с первого элемента.

ПП ПМ.12 · Раздел 33

Адаптация · Доставка

1

Что хранить

Подбор и доставка — два разных события. hasCargo хранит наличие груза, delivered — число доставок.

2

Как изменить логику

Подбирать предмет только при hasCargo = false. При входе в нужную зону проверять hasCargo, затем увеличить delivered и сбросить hasCargo. Повторный вход пустым не меняет счёт.

3

Место изменения и порядок обработки

В собственном обработчике подбора нужны cargoId, targetZoneId и hasCargo. Разрешить подбор только при Playing и hasCargo == false; запомнить исходную позицию предмета, отключить его, установить hasCargo. Счёт не менять. В обработчике триггера зоны: при !hasCargo выйти; при несовпадении targetZoneId оставить груз и вывести причину; при совпадении увеличить delivered, очистить груз. Адресат задаётся в записи предмета: для 2–3 зон распределить ID так, чтобы у каждой доставки существовал адресат и доступный маршрут. В варианте 14 урон возвращает отключённую аптечку в сохранённую точку и включает её до очистки cargoId. При повторе восстановить все предметы и hasCargo=false. HUD отдельно показывает груз и delivered/норму.

4

Уровни и сброс

При повреждении поведение груза определяется вариантом. Для спасательного дрона вернуть аптечку в исходную точку, прежде чем очистить ссылку на груз.

5

Проверка границы

Проверить вторую попытку подбора, неверную зону доставки, повреждение с грузом и повтор уровня.

ПП ПМ.12 · Раздел 34

Адаптация · Вместимость

1

Что хранить

capacity — предел груза, carried — текущий груз, delivered — итог доставки. Эти числа нельзя объединять в один счётчик.

2

Как изменить логику

Подбирать предмет при carried < capacity. Разгрузка выполняет delivered += carried, затем carried = 0. Предмет сверх лимита оставить активным, чтобы его можно было взять после разгрузки.

3

Место изменения и порядок обработки

В данных уровня хранить total и capacity, в попытке carried=0 и delivered=0. Обработчик контакта: если Playing и carried < capacity, записать ID, отключить предмет, увеличить carried; иначе оставить предмет доступным, вывести «Нет места». В разгрузке выполнить delivered += carried, затем carried=0; при нуле изменений нет. Условие выхода: delivered >= total, а не carried + delivered до разгрузки. Вариант 17 показывает 0/2, 1/2, 2/2; вариант 19 одновременно число и шкалу carried/capacity. Округление шкалы не меняет вместимость. Нулевой кислород, энергия или батарея вызывает Lost до проверки победы; начальный ресурс и расход хранить отдельно от груза. При повторе восстановить ID и оба счётчика.

4

Уровни и сброс

В данных уровня задать число предметов и capacity. При старте сбросить оба счётчика. В постоянный рекорд включать только успешно доставленное.

5

Проверка границы

Разгрузить неполный рюкзак: должно добавиться реальное carried, а не capacity. Повторно разгрузить пустой: счёт не изменяется.

ПП ПМ.12 · Раздел 35

Адаптация · Комплект предметов

1

Что хранить

Для уникальных предметов использовать отдельные логические флаги; для количества — отдельные целые счётчики. Общая сумма не доказывает комплектность.

2

Как изменить логику

Уникальный предмет меняет свой флаг только один раз. Для количественного типа увеличивать счёт до нужной нормы. Перед выходом проверить каждую норму отдельно.

3

Место изменения и порядок обработки

В собственных данных уровня задать required[] по типам. Для варианта 21 нормы травы/кристалла равны 2/1, 3/2, 4/3; для 25 норма образцов 2,3,4 и реагент 1. Для уникальных 22–24 использовать флаги либо нормы 1 на каждый тип. В попытке хранить count[] и набор ID. После правильного контакта увеличивать только count[type] и не выше required[type]; чужой тип и дубликат не заменяют отсутствующий. Выход разрешить лишь если каждый count[i] >= required[i]. Для стола, ремонта и анализатора проверка выполняется в соответствующей зоне. HUD показывает каждый тип отдельно. Сброс обнуляет весь массив и вновь включает предметы. Особый тест: избыток первого типа при отсутствии последнего не даёт победу.

4

Уровни и сброс

В вариантах 22–24 предметы остаются уникальными на всех уровнях: усложнять маршрут и помехи. В варианте 25 менять число образцов, а реагент сохранять отдельным флагом.

5

Проверка границы

Собрать избыток одного типа без другого: выход остаётся закрытым. На повторе все флаги и счётчики сброшены.

ПП ПМ.12 · Раздел 36

Адаптация · Открытие проходов

1

Что хранить

Флаг описывает состояние отдельного ключа или переключателя. Связь между активатором и воротами должна быть явной.

2

Как изменить логику

Ключ открывает только свои ворота. Терминал отключает Collider2D своей стены. Для камеры спасательной системы проверять одновременно два разных флага, а не число нажатий.

3

Место изменения и порядок обработки

В записи уровня хранить связи activatorId → barrierId. Для каждого активатора нужен отдельный bool; повтор true не меняет прогресс. Ключ открывает лишь свои ворота: после верного ключа Collider2D ворот выключить и изменить вид. Терминал выключает только Collider2D назначенной стены. Мост становится проходимым: включить видимую поверхность и её безопасный коллайдер/зону, убрать опасную область этого участка; одного скрытия стены недостаточно. Для камер варианта 30 хранить два различных switchId на каждую камеру: дверь открывается по флагам обоих, затем робот должен достигнуть выхода; открытие двери ещё не является спасением. При повторе включить закрытые барьеры, убрать мосты, сбросить флаги и вернуть роботов. Проверить фактическое прохождение, ошибочный ID и два нажатия одного переключателя.

4

Уровни и сброс

После открытия ворота становятся проходимыми. При новой попытке восстановить коллайдеры и сбросить флаги. Постоянно хранить открытый уровень, а не временное положение ворот.

5

Проверка границы

Активировать один рубильник дважды: второй флаг остаётся false. Войти в открывшийся проход и проверить физическую проходимость.

ПП ПМ.12 · Раздел 37

Код · LevelManager.cs

1

Назначение

Хранит параметры и проверяет доступность уровня.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 38

LevelManager.cs · строки 1–33

1

Код файла

```
using UnityEngine;

[System.Serializable]
public class LevelData
{
    public string title;
    [Range(1, 7)] public int target;
    [Range(1, 6)] public int hazards;
    [Min(20)] public float seconds;
}

public class LevelManager : MonoBehaviour
{
    public LevelData[] levels =
    {
        new() { title = "Первый маршрут", target = 3,
            hazards = 2, seconds = 90 },
        new() { title = "Опасный поворот", target = 5,
            hazards = 4, seconds = 80 },
        new() { title = "Последний портал", target = 7,
            hazards = 6, seconds = 70 }
    };
    public int Current { get; private set; }
    public LevelData Data => levels[Current - 1];

    public bool Select(int number)
    {
        if (number < 1 || number > levels.Length ||
            number > SaveService.Unlocked) return false;
        Current = number;
        return true;
    }
}
```

ПП ПМ.12 · Раздел 39

Код · SaveService.cs

1

Назначение

Хранит прогресс, рекорды и громкость между запусками.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

SaveService.cs · строки 1–31

1

Код файла

```
using UnityEngine;

public static class SaveService
{
    // Свой префикс не затрагивает сохранения других учебных проектов.
    private const string Prefix = "PP12.Crystal.v1.";
    public static int Unlocked => Mathf.Clamp(
        PlayerPrefs.GetInt(Prefix + "Unlocked", 1), 1, 3);
    public static float Volume => Mathf.Clamp01(
        PlayerPrefs.GetFloat(Prefix + "Volume", 0.7f));

    public static int Best(int level) => Mathf.Max(0,
        PlayerPrefs.GetInt(Prefix + "Best" + level, 0));

    public static void Complete(int level, int score)
    {
        PlayerPrefs.SetInt(Prefix + "Unlocked",
            Mathf.Max(Unlocked, Mathf.Min(level + 1, 3)));
        if (score > Best(level))
            PlayerPrefs.SetInt(Prefix + "Best" + level, score);
        PlayerPrefs.Save();
    }

    public static void SetVolume(float value)
    {
        value = Mathf.Clamp01(value);
        AudioListener.volume = value;
        PlayerPrefs.SetFloat(Prefix + "Volume", value);
        PlayerPrefs.Save();
    }
}
```

ПП ПМ.12 · Раздел 41

Код · GameManager.cs

1

Назначение

Управляет попыткой, состояниями, победой и поражением.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

GameManager.cs · строки 1–28

1

Код файла

```
using UnityEngine;

[DefaultExecutionOrder(-50)]
[RequireComponent(typeof(LevelManager), typeof(WorldBuilder))]
public class GameManager : MonoBehaviour
{
    public enum State { Menu, Playing, Paused, Won, Lost }
    public static GameManager Instance { get; private set; }
    public State CurrentState { get; private set; } = State.Menu;
    public bool IsPlaying => CurrentState == State.Playing;
    public Vector2 MobileDirection { get; private set; }
    public int Collected { get; private set; }
    public int Hearts { get; private set; }
    public float Remaining { get; private set; }
    public string Hint { get; private set; } = "Выберите уровень";
    public LevelManager Levels { get; private set; }
    private WorldBuilder world;
    private AudioFeedback audioFeedback;
    private float immuneUntil;
    private float elapsed;

    private void Awake()
    {
        Instance = this;
        Levels = GetComponent<LevelManager>();
        world = GetComponent<WorldBuilder>();
        audioFeedback = gameObject.AddComponent<AudioFeedback>();
        gameObject.AddComponent<PauseController>();
```

ПП ПМ.12 · Раздел 43

GameManager.cs · строки 29–56

1

Код файла

```
        gameObject.AddComponent<MenuController>();
        Application.targetFrameRate = 60;
        AudioListener.volume = SaveService.Volume;
        Time.timeScale = 1;
        world.BuildCamera();
    }

    public void StartLevel(int number)
    {
        if (!Levels.Select(number)) return;
        Time.timeScale = 1;
        Collected = 0;
        Hearts = 3;
        Remaining = Levels.Data.seconds;
        elapsed = 0;
        immuneUntil = 0;
        MobileDirection = Vector2.zero;
        Hint = "Соберите все кристаллы и войдите в портал";
        world.Build(Levels.Data);
        CurrentState = State.Playing;
    }

    private void Update()
    {
        if (!IsPlaying) return;
        elapsed += Time.deltaTime;
        Remaining = Mathf.Max(0, Remaining - Time.deltaTime);
        if (Remaining <= 0) Finish(false, "Время закончилось");
```

GameManager.cs · строки 57–83

1

Код файла

```
}

public void SetDirection(Vector2 value) => MobileDirection = value;

public void Collect(Collectible item)
{
    if (!IsPlaying || !item.gameObject.activeSelf) return;
    item.gameObject.SetActive(false);
    Collected++;
    audioFeedback.PlayCollect();
    Hint = Collected == Levels.Data.target
        ? "Все кристаллы собраны. Войдите в зелёный портал"
        : $"Собрано {Collected} из {Levels.Data.target}";
}

public void HitHazard()
{
    if (!IsPlaying || Time.time < immuneUntil) return;
    Hearts--;
    immuneUntil = Time.time + 1;
    audioFeedback.PlayHazard();
    world.Player.ReturnToStart();
    Hint = "Потеряна жизнь. Измените маршрут";
    if (Hearts <= 0) Finish(false, "Жизни закончились");
}

public void TryFinishLevel()
```

ПП ПМ.12 · Раздел 45

GameManager.cs · строки 84–111

1

Код файла

```
{
    if (!IsPlaying) return;
    if (Collected < Levels.Data.target)
    {
        Hint = $"Нужно ещё {Levels.Data.target - Collected} кристаллов";
        return;
    }
    int score = Mathf.Max(1, 1000 + Hearts * 100 -
        Mathf.RoundToInt(elapsed * 5));
    SaveService.Complete(Levels.Current, score);
    audioFeedback.PlaySuccess();
    Finish(true, $"Очки: {score}. Рекорд: {SaveService.Best(Levels.Current)}");
}

private void Finish(bool won, string message)
{
    CurrentState = won ? State.Won : State.Lost;
    Hint = message;
    MobileDirection = Vector2.zero;
    Time.timeScale = 0;
}

public void Pause()
{
    if (!IsPlaying) return;
    CurrentState = State.Paused;
    MobileDirection = Vector2.zero;
    Time.timeScale = 0;
```

ПП ПМ.12 · Раздел 46

GameManager.cs · строки 112–139

1

Код файла

```
        PlayerPrefs.Save();
    }

    public void TogglePause()
    {
        if (IsPlaying) Pause();
        else if (CurrentState == State.Paused)
        {
            CurrentState = State.Playing;
            Time.timeScale = 1;
        }
    }

    public void ToMenu()
    {
        CurrentState = State.Menu;
        MobileDirection = Vector2.zero;
        Time.timeScale = 1;
        world.Clear();
        PlayerPrefs.Save();
    }

    private void OnDestroy()
    {
        if (Instance == this) Instance = null;
        Time.timeScale = 1;
    }
}
```

ПП ПМ.12 · Раздел 47

Код · MenuController.cs

1

Назначение

Создаёт UI и связывает кнопки с действиями игры.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 48

MenuController.cs · строки 1–30

1

Код файла

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
using UnityEngine.EventSystems;

public class MenuController : MonoBehaviour
{
    private GameManager game;
    private GameObject menu, hud, pause, result, settings;
    private Text progress, hint, resultTitle, resultText, volumeText;
    [SerializeField] private Button[] levels = new Button[3];
    private Button next;
    private Font font;
    private Transform canvas;

    private void Start()
    {
        game = GameManager.Instance;
        font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        var go = new GameObject("Canvas", typeof(Canvas),
            typeof(CanvasScaler), typeof(GraphicRaycaster));
        canvas = go.transform;
        go.GetComponent<Canvas>().renderMode = RenderMode.ScreenSpaceOverlay;
        var scaler = go.GetComponent<CanvasScaler>();
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(1280, 720);
        scaler.matchWidthOrHeight = 0.5f;
        if (FindAnyObjectByType<EventSystem>() == null)
            new GameObject("EventSystem", typeof(EventSystem),
                typeof(StandaloneInputModule));
```

MenuController.cs · строки 31–60

1

Код файла

```
        BuildMenu();
        BuildHud();
        BuildOverlays();
    }

    private void BuildMenu()
    {
        menu = Panel("MainMenuPanel", canvas, 0, 0, 860, 550);
        Label(menu.transform, "КРИСТАЛЬНЫЙ МАРШРУТ", 0, 205, 800, 50, 34);
        Label(menu.transform, "Три маршрута. Соберите кристаллы и найдите выход.",
            0, 140, 800, 40, 22);
        for (int i = 0; i < 3; i++)
        {
            int number = i + 1;
            levels[i] = Button(menu.transform, "Level" + number,
                "", 0, 60 - i * 85, 640, 66, () => game.StartLevel(number));
        }
        Button(menu.transform, "Settings", "Настройки звука", 0, -215,
            420, 55, () => ShowSettings(true));
    }

    private void BuildHud()
    {
        hud = new GameObject("GamePanel", typeof(RectTransform));
        hud.transform.SetParent(canvas, false);
        var bar = Panel("HUDPanel", hud.transform, 0, 305, 1230, 80);
        progress = Label(bar.transform, "", -70, 0, 1020, 55, 24);
        Button(bar.transform, "PauseButton", "Пауза", 525, 0, 160, 54, game.Pause);
        hint = Label(hud.transform, "", 120, -305, 820, 50, 21);
        Direction("Left", "←", -565, -295, Vector2.left);
```

MenuController.cs · строки 61–90

1

Код файла

```
        Direction("Down", "↓", -475, -295, Vector2.down);
        Direction("Right", "→", -385, -295, Vector2.right);
        Direction("Up", "↑", -475, -205, Vector2.up);
    }

    private void Direction(string name, string label, float x, float y, Vector2 d)
    {
        var button = Button(hud.transform, name, label, x, y, 80, 80, null);
        button.gameObject.AddComponent<HoldButton>().direction = d;
    }

    private void BuildOverlays()
    {
        pause = Panel("PausePanel", canvas, 0, 0, 700, 360);
        Label(pause.transform, "ПАУЗА", 0, 115, 620, 60, 36);
        Button(pause.transform, "Resume", "Продолжить", 0, 15, 420, 65,
            game.TogglePause);
        Button(pause.transform, "Menu", "В меню", 0, -80, 420, 65, game.ToMenu);
        result = Panel("ResultPanel", canvas, 0, 0, 780, 450);
        resultTitle = Label(result.transform, "", 0, 165, 740, 55, 32);
        resultText = Label(result.transform, "", 0, 95, 740, 50, 22);
        next = Button(result.transform, "Next", "Следующий уровень", 0, 10,
            480, 65, () => game.StartLevel(game.Levels.Current + 1));
        Button(result.transform, "Retry", "Повторить уровень", 0, -75,
            480, 65, () => game.StartLevel(game.Levels.Current));
        Button(result.transform, "Menu", "В меню", 0, -160, 480, 65, game.ToMenu);
        settings = Panel("SettingsPanel", canvas, 0, 0, 780, 450);
        Label(settings.transform, "НАСТРОЙКИ", 0, 160, 700, 60, 34);
        volumeText = Label(settings.transform, "", 0, 70, 700, 50, 28);
        Button(settings.transform, "Minus", "−", -140, -20, 120, 65,
```

MenuController.cs · строки 91–121

1

Код файла

```
            () => SaveService.SetVolume(SaveService.Volume - 0.1f));
        Button(settings.transform, "Plus", "+", 140, -20, 120, 65,
            () => SaveService.SetVolume(SaveService.Volume + 0.1f));
        Button(settings.transform, "Back", "Назад", 0, -140, 420, 65,
            () => ShowSettings(false));
        settings.SetActive(false);
    }

    private void ShowSettings(bool show)
    {
        settings.SetActive(show);
        menu.SetActive(!show);
    }

    private void Update()
    {
        if (game == null) return;
        var state = game.CurrentState;
        menu.SetActive(state == GameManager.State.Menu && !settings.activeSelf);
        hud.SetActive(state != GameManager.State.Menu);
        pause.SetActive(state == GameManager.State.Paused);
        bool ended = state == GameManager.State.Won || state ==
GameManager.State.Lost;
        result.SetActive(ended);
        if (state != GameManager.State.Menu) settings.SetActive(false);
        for (int i = 0; i < 3; i++)
        {
            bool unlocked = i + 1 <= SaveService.Unlocked;
            levels[i].interactable = unlocked;
            levels[i].GetComponentInChildren<Text>().text = unlocked
                ? $"Уровень {i + 1}  ·  рекорд {SaveService.Best(i + 1)}"
                : $"Уровень {i + 1}  ·  закрыт";
```

ПП ПМ.12 · Раздел 52

MenuController.cs · строки 122–151

1

Код файла

```
        }
        volumeText.text = $"Громкость: {Mathf.RoundToInt(SaveService.Volume *
100)}%";
        if (game.Levels.Current == 0) return;
        progress.text = $"Уровень {game.Levels.Current}   Кристаллы " +
            $"{game.Collected}/{game.Levels.Data.target}   Жизни {game.Hearts}" +
            $"   Время {Mathf.CeilToInt(game.Remaining)}";
        hint.text = game.Hint;
        resultTitle.text = state == GameManager.State.Won
            ? (game.Levels.Current == 3 ? "ИГРА ПРОЙДЕНА" : "УРОВЕНЬ ПРОЙДЕН")
            : "ПОПРОБУЙТЕ ЕЩЁ";
        resultText.text = game.Hint;
        next.gameObject.SetActive(state == GameManager.State.Won &&
game.Levels.Current < 3);
    }

    private GameObject Panel(string name, Transform parent,
        float x, float y, float width, float height)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        Place(go, parent, x, y, width, height);
        go.GetComponent<Image>().color = VisualTheme.Panel;
        return go;
    }

    private Text Label(Transform parent, string value, float x, float y,
        float width, float height, int size)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        Place(go, parent, x, y, width, height);
        var text = go.GetComponent<Text>();
        text.font = font;
```

ПП ПМ.12 · Раздел 53

MenuController.cs · строки 152–181

1

Код файла

```
        text.fontSize = size;
        text.color = Color.white;
        text.alignment = TextAnchor.MiddleCenter;
        text.raycastTarget = false;
        text.text = value;
        return text;
    }

    private Button Button(Transform parent, string name, string label,
        float x, float y, float width, float height, UnityAction action)
    {
        var go = Panel(name, parent, x, y, width, height);
        go.GetComponent<Image>().color = VisualTheme.Accent;
        var button = go.AddComponent<Button>();
        button.targetGraphic = go.GetComponent<Image>();
        if (action != null) button.onClick.AddListener(action);
        Label(go.transform, label, 0, 0, width - 12, height - 8, 24);
        return button;
    }

    private void Place(GameObject go, Transform parent,
        float x, float y, float width, float height)
    {
        go.transform.SetParent(parent, false);
        var rt = go.GetComponent<RectTransform>();
        rt.anchorMin = rt.anchorMax = rt.pivot = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(x, y);
        rt.sizeDelta = new Vector2(width, height);
    }
}
```

ПП ПМ.12 · Раздел 54

Код · PauseController.cs

1

Назначение

Реагирует на Escape, потерю фокуса и приостановку приложения.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 55

PauseController.cs · строки 1–27

1

Код файла

```
using UnityEngine;

public class PauseController : MonoBehaviour
{
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
            GameManager.Instance.TogglePause();
    }

    private void OnApplicationFocus(bool hasFocus)
    {
        if (!hasFocus) GameManager.Instance?.Pause();
    }

    private void OnApplicationPause(bool paused)
    {
        if (paused) GameManager.Instance?.Pause();
        PlayerPrefs.Save();
    }

    // Возврат фокуса не возобновляет игру без действия игрока.
    private void OnDisable()
    {
        Time.timeScale = 1f;
    }
}
```

ПП ПМ.12 · Раздел 56

Код · WorldBuilder.cs

1

Назначение

Создаёт поле по данным уровня и повторно использует спрайты.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 57

WorldBuilder.cs · строки 1–28

1

Код файла

```
using System.Collections.Generic;
using UnityEngine;

public class WorldBuilder : MonoBehaviour
{
    public PlayerController Player { get; private set; }
    private GameObject root;
    private readonly Dictionary<int, Sprite> sprites = new();
    private readonly Vector2[] crystals =
    {
        new(-5.7f, 2.8f), new(-2.5f, 0.8f), new(0.8f, 3),
        new(3.2f, -1.8f), new(5.7f, 2.2f), new(-1, -3), new(6, -3)
    };
    private readonly Vector2[] hazards =
    {
        new(-4, -1.8f), new(-0.6f, -0.2f), new(2.2f, 1.5f),
        new(5, -0.5f), new(-6, 0), new(0, -3.5f)
    };

    public void BuildCamera()
    {
        Camera cam = Camera.main;
        if (cam == null)
        {
            var go = new GameObject("Main Camera");
            go.tag = "MainCamera";
            cam = go.AddComponent<Camera>();
        }
```

WorldBuilder.cs · строки 29–56

1

Код файла

```
        cam.orthographic = true;
        cam.orthographicSize = 6.2f;
        cam.transform.position = new Vector3(0, 0, -10);
        cam.clearFlags = CameraClearFlags.SolidColor;
        cam.backgroundColor = VisualTheme.Background;
        if (!cam.GetComponent<AudioListener>())
            cam.gameObject.AddComponent<AudioListener>();
    }

    public void Clear()
    {
        if (!root) return;
        root.SetActive(false);
        Destroy(root);
    }

    public void Build(LevelData data)
    {
        Clear();
        root = new GameObject("LevelObjects");
        Item("Field", Vector2.zero, new(17, 9.5f), VisualTheme.Surface, 0, -10);
        Wall("Top", new(0, 4.7f), new(17, 0.3f));
        Wall("Bottom", new(0, -4.7f), new(17, 0.3f));
        Wall("Left", new(-8.4f, 0), new(0.3f, 9.6f));
        Wall("Right", new(8.4f, 0), new(0.3f, 9.6f));
        Wall("A", new(-3.8f, 1.2f), new(0.35f, 3));
        if (data.target >= 5)
            Wall("B", new(1.4f, -2.2f), new(3.2f, 0.35f));
```

ПП ПМ.12 · Раздел 59

WorldBuilder.cs · строки 57–84

1

Код файла

```
        if (data.target >= 7)
            Wall("C", new(4.4f, 2), new(0.35f, 2.5f));
        var player = Item("Player", new(-6.5f, -3.1f), new(0.7f, 0.7f),
            VisualTheme.Player, 1, 10);
        player.AddComponent<Rigidbody2D>();
        player.AddComponent<CircleCollider2D>();
        Player = player.AddComponent<PlayerController>();
        for (int i = 0; i < Mathf.Clamp(data.target, 1, crystals.Length); i++)
        {
            var c = Item("Crystal" + (i + 1), crystals[i], new(0.6f, 0.8f),
                VisualTheme.Collectible, 2, 5);
            c.AddComponent<PolygonCollider2D>();
            c.AddComponent<CollectibleVisual>();
            c.AddComponent<Collectible>();
        }
        for (int i = 0; i < Mathf.Clamp(data.hazards, 1, hazards.Length); i++)
        {
            var h = Item("Hazard" + (i + 1), hazards[i], new(0.8f, 0.8f),
                VisualTheme.Danger, 3, 4);
            h.AddComponent<CircleCollider2D>();
            h.AddComponent<Hazard>();
        }
        var exit = Item("Exit", new(6.5f, 3.1f), new(1, 1),
            VisualTheme.Success, 4, 2);
        exit.AddComponent<BoxCollider2D>();
        exit.AddComponent<ExitZone>();
    }
```

ПП ПМ.12 · Раздел 60

WorldBuilder.cs · строки 85–112

1

Код файла

```
private void Wall(string name, Vector2 p, Vector2 scale)
{
    Item("Wall" + name, p, scale, VisualTheme.Button, 0, 1)
        .AddComponent<BoxCollider2D>();
}

private GameObject Item(string name, Vector2 p, Vector2 scale,
    Color color, int shape, int order)
{
    var go = new GameObject(name);
    go.transform.SetParent(root.transform);
    go.transform.position = p;
    go.transform.localScale = scale;
    var sr = go.AddComponent<SpriteRenderer>();
    sr.sprite = GetSprite(shape);
    sr.color = color;
    sr.sortingOrder = order;
    return go;
}

private Sprite GetSprite(int shape)
{
    if (sprites.TryGetValue(shape, out Sprite cached)) return cached;
    const int size = 64;
    var texture = new Texture2D(size, size);
    for (int y = 0; y < size; y++)
    for (int x = 0; x < size; x++)
    {
```

ПП ПМ.12 · Раздел 61

WorldBuilder.cs · строки 113–140

1

Код файла

```
            float a = (x + 0.5f) / size * 2 - 1;
            float b = (y + 0.5f) / size * 2 - 1;
            bool inside = shape switch
            {
                1 => a * a + b * b < 0.9f,
                2 => Mathf.Abs(a) + Mathf.Abs(b) < 0.92f,
                3 => Mathf.Abs(a - b) < 0.2f || Mathf.Abs(a + b) < 0.2f,
                4 => Mathf.Max(Mathf.Abs(a), Mathf.Abs(b)) > 0.65f,
                _ => true
            };
            texture.SetPixel(x, y, inside ? Color.white : Color.clear);
        }
        texture.Apply();
        var sprite = Sprite.Create(texture, new Rect(0, 0, size, size),
            new Vector2(0.5f, 0.5f), size);
        sprites[shape] = sprite;
        return sprite;
    }

    private void OnDestroy()
    {
        foreach (Sprite sprite in sprites.Values)
        {
            Destroy(sprite.texture);
            Destroy(sprite);
        }
    }
}
```

ПП ПМ.12 · Раздел 62

Код · HoldButton.cs

1

Назначение

Передаёт направление при удержании и сбрасывает его при отпускании.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 63

HoldButton.cs · строки 1–24

1

Код файла

```
using UnityEngine;
using UnityEngine.EventSystems;

public class HoldButton : MonoBehaviour,
    IPointerDownHandler, IPointerUpHandler, IPointerExitHandler
{
    public Vector2 direction;
    private bool held;
    public void OnPointerDown(PointerEventData e)
    {
        held = true;
        if (GameManager.Instance.IsPlaying)
            GameManager.Instance.SetDirection(direction);
    }
    public void OnPointerUp(PointerEventData e) => Release();
    public void OnPointerExit(PointerEventData e) => Release();
    private void OnDisable() => Release();
    private void Release()
    {
        if (held && GameManager.Instance != null)
            GameManager.Instance.SetDirection(Vector2.zero);
        held = false;
    }
}
```

ПП ПМ.12 · Раздел 64

Код · PlayerController.cs

1

Назначение

Двигает игрока и останавливает его вне Playing.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 65

PlayerController.cs · строки 1–24

1

Код файла

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody2D), typeof(Collider2D))]
public class PlayerController : MonoBehaviour
{
    [SerializeField, Min(1f)] private float speed = 5f;
    private Rigidbody2D body;
    private Vector2 startPosition;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        body.gravityScale = 0f;
        body.freezeRotation = true;
        startPosition = transform.position;
    }

    private void FixedUpdate()
    {
        if (GameManager.Instance == null || !GameManager.Instance.IsPlaying)
        {
            body.linearVelocity = Vector2.zero;
            return;
        }
```

ПП ПМ.12 · Раздел 66

PlayerController.cs · строки 25–49

1

Код файла

```
        Vector2 keyboard = new Vector2(
            ReadAxis(KeyCode.A, KeyCode.LeftArrow, KeyCode.D, KeyCode.RightArrow),
            ReadAxis(KeyCode.S, KeyCode.DownArrow, KeyCode.W, KeyCode.UpArrow));
        Vector2 direction = Vector2.ClampMagnitude(
            keyboard + GameManager.Instance.MobileDirection, 1f);
        body.linearVelocity = direction * speed;
    }

    private static float ReadAxis(
        KeyCode negativeA, KeyCode negativeB,
        KeyCode positiveA, KeyCode positiveB)
    {
        float value = 0f;
        if (Input.GetKey(negativeA) || Input.GetKey(negativeB)) value -= 1f;
        if (Input.GetKey(positiveA) || Input.GetKey(positiveB)) value += 1f;
        return value;
    }

    public void ReturnToStart()
    {
        body.linearVelocity = Vector2.zero;
        body.position = startPosition;
    }
}
```

ПП ПМ.12 · Раздел 67

Код · Collectible.cs

1

Назначение

Передаёт событие сбора менеджеру.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 68

Collectible.cs · строки 1–16

1

Код файла

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class Collectible : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.Collect(this);
    }
}
```

ПП ПМ.12 · Раздел 69

Код · Hazard.cs

1

Назначение

Передаёт событие столкновения с опасностью.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 70

Hazard.cs · строки 1–16

1

Код файла

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class Hazard : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.HitHazard();
    }
}
```

ПП ПМ.12 · Раздел 71

Код · ExitZone.cs

1

Назначение

Проверяет завершение при входе в портал.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 72

ExitZone.cs · строки 1–16

1

Код файла

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class ExitZone : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.TryFinishLevel();
    }
}
```

ПП ПМ.12 · Раздел 73

Код · VisualTheme.cs

1

Назначение

Задаёт палитру контрольной игры.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 74

VisualTheme.cs · строки 1–16

1

Код файла

```
using UnityEngine;

public static class VisualTheme
{
    public static readonly Color Background = new(0.035f, 0.055f, 0.10f);
    public static readonly Color Surface = new(0.07f, 0.10f, 0.18f);
    public static readonly Color Panel = new(0.055f, 0.075f, 0.13f, 0.96f);
    public static readonly Color Button = new(0.12f, 0.18f, 0.30f, 0.94f);
    public static readonly Color Accent = new(0.16f, 0.55f, 0.75f, 0.98f);
    public static readonly Color Player = new(0.19f, 0.88f, 1f);
    public static readonly Color Collectible = new(1f, 0.78f, 0.16f);
    public static readonly Color Danger = new(1f, 0.25f, 0.28f);
    public static readonly Color Success = new(0.25f, 0.82f, 0.45f);
    public static readonly Color PrimaryText = Color.white;
    public static readonly Color SecondaryText = new(0.82f, 0.88f, 0.98f);
}
```

ПП ПМ.12 · Раздел 75

Код · AudioFeedback.cs

1

Назначение

Создаёт и воспроизводит звуковую обратную связь.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 76

AudioFeedback.cs · строки 1–34

1

Код файла

```
using UnityEngine;

[RequireComponent(typeof(AudioSource))]
public class AudioFeedback : MonoBehaviour
{
    private AudioSource source;
    private AudioClip collectClip;
    private AudioClip hazardClip;
    private AudioClip successClip;

    private void Awake()
    {
        EnsureReady();
    }

    private void EnsureReady()
    {
        if (source == null)
        {
            source = GetComponent<AudioSource>();
            source.playOnAwake = false;
        }

        collectClip ??= CreateTone("Collect", 660f, 0.10f, 0.20f);
        hazardClip ??= CreateTone("Hazard", 180f, 0.18f, 0.24f);
        successClip ??= CreateTone("Success", 880f, 0.30f, 0.18f);
    }

    public void PlayCollect()
    {
        EnsureReady();
        source.PlayOneShot(collectClip);
    }
```

AudioFeedback.cs · строки 35–67

1

Код файла

```
    public void PlayHazard()
    {
        EnsureReady();
        source.PlayOneShot(hazardClip);
    }

    public void PlaySuccess()
    {
        EnsureReady();
        source.PlayOneShot(successClip);
    }

    private static AudioClip CreateTone(string clipName, float frequency,
        float duration, float volume)
    {
        const int sampleRate = 44100;
        int sampleCount = Mathf.CeilToInt(sampleRate * duration);
        float[] samples = new float[sampleCount];

        for (int i = 0; i < sampleCount; i++)
        {
            float time = (float)i / sampleRate;
            float fade = 1f - (float)i / sampleCount;
            samples[i] = Mathf.Sin(2f * Mathf.PI * frequency * time)
                * volume * fade;
        }

        AudioClip clip = AudioClip.Create(clipName, sampleCount, 1,
            sampleRate, false);
        clip.SetData(samples, 0);
        return clip;
    }
}
```

ПП ПМ.12 · Раздел 78

Код · CollectibleVisual.cs

1

Назначение

Показывает вращение и пульсацию предметов.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 79

CollectibleVisual.cs · строки 1–27

1

Код файла

```
using UnityEngine;

public class CollectibleVisual : MonoBehaviour
{
    [SerializeField, Min(0f)] private float rotationSpeed = 70f;
    [SerializeField, Range(0f, 0.25f)] private float pulseAmount = 0.08f;
    [SerializeField, Min(0.1f)] private float pulseSpeed = 4f;

    private Vector3 startScale;

    private void Awake()
    {
        startScale = transform.localScale;
    }

    private void Update()
    {
        transform.Rotate(0f, 0f, rotationSpeed * Time.deltaTime);
        float pulse = 1f + Mathf.Sin(Time.time * pulseSpeed) * pulseAmount;
        transform.localScale = startScale * pulse;
    }

    private void OnDisable()
    {
        transform.localScale = startScale;
    }
}
```

ПП ПМ.12 · Раздел 80

Код · GameBootstrap.cs

1

Назначение

Создаёт менеджер, если его ещё нет в сцене.

2

Как вставлять

Открыть файл с этим именем. Заменить его целиком всеми следующими частями по порядку, без заголовков и номеров строк. Сохранить только после последней части. Полный файл также доступен отдельно в загрузках.

ПП ПМ.12 · Раздел 81

GameBootstrap.cs · строки 1–14

1

Код файла

```
using UnityEngine;

public static class GameBootstrap
{
    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    private static void CreateGame()
    {
        if (Object.FindAnyObjectByType<GameManager>() != null)
            return;

        var root = new GameObject("CrystalQuest");
        root.AddComponent<GameManager>();
    }
}
```